

QuickClips

Piyush Solanki

Roll no. 150096724026

Semester IV | B.Tech CSE (2024–2028)

1. Problem Statement

The Challenge

QuickClips is a short-form video sharing platform designed to handle high-concurrency content uploads and real-time personalized video delivery. The system faces three critical challenges:

- Video Playback Latency:** Users expect instant playback with zero buffering when scrolling. Payloads uploaded directly to APIs introduce load lag.
- Personalization Inefficiency:** Scanning relational tables across millions of rows to generate personalized feeds in real-time creates computational bottlenecks on APIs.
- High Write Throughput:** Massive volumes of continuous user interactions (likes, shares, views) create write contention that easily saturates traditional databases.

2. Proposed Solution

Proposed Solution

Develop a comprehensive, scalable system design modeling a decoupled microservices architecture on AWS:

- Asynchronous Transcoding:** Decouple upload payloads via message queues (RabbitMQ/Kafka) consumed by FFmpeg worker instances for 360p/720p/1080p HLS encoding.
- Global Edge Caching:** Store manifest playlists on AWS S3 and serve them globally via CloudFront CDN edges to reduce playback buffering to sub-100ms.
- Hybrid Personalization:** Compute user feeds using a three-signal algorithm (content affinity, social graph, and trending velocity) in Python and cached via Redis.
- Write Path Decoupling:** Store heavy interaction log streams in Cassandra, while using Redis atomic operations for like/view counter updates.

3. System Architecture

3.1 System Overview

The system is structured as a decoupled, multi-layered architecture where each service operates independently and scales horizontally. Clients interact exclusively with stateless web services behind a load balancer, while background workloads are deferred to message queue consumers. The storage tier uses a polyglot persistence design, matching specific database properties to transactional, caching, and analytical logging workloads.

3.2 High-Level System Architecture Diagram

The architectural blueprint below illustrates the six distinct system layers: Client, API Gateway, Microservices, Event Broker, Data/Persistence, and Delivery (CDN).

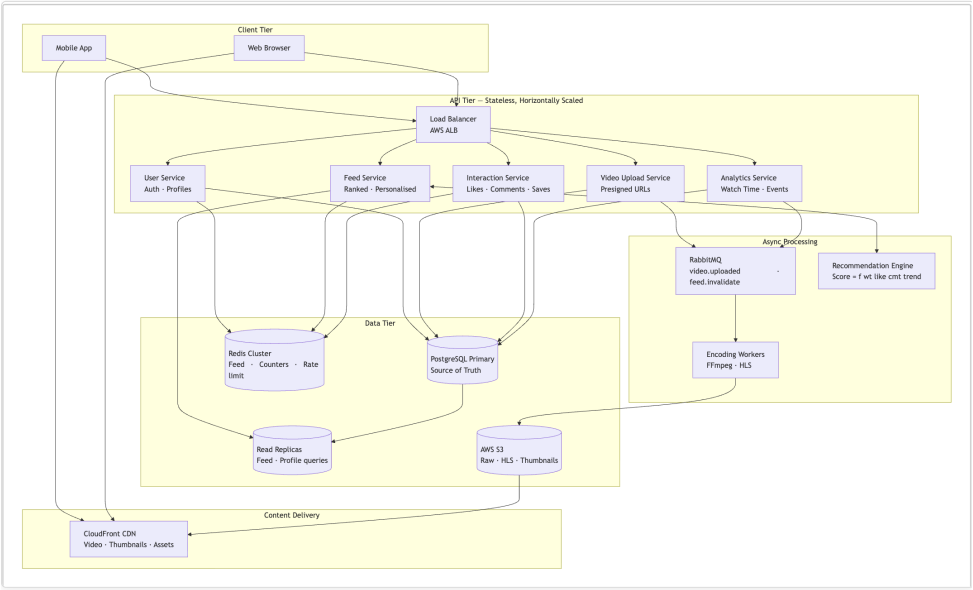
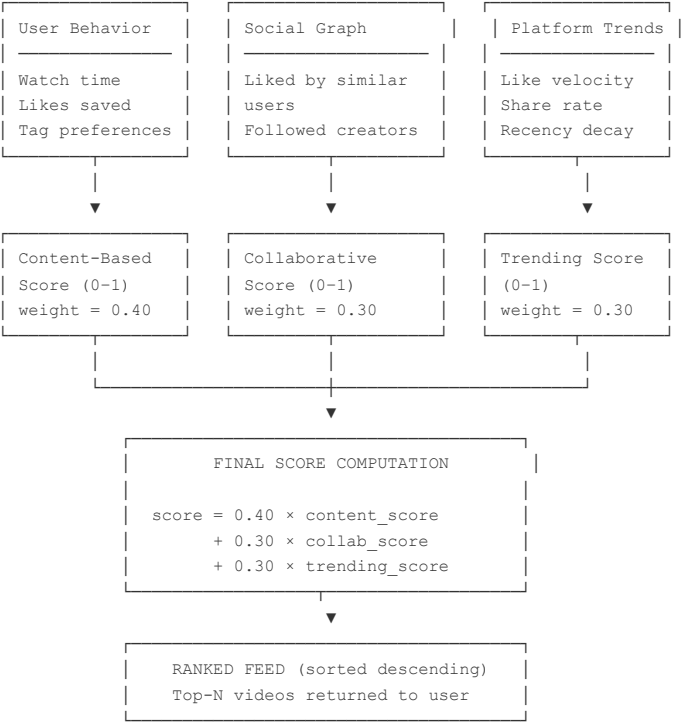


Figure 1 — QuickClips High-Level System Architecture Diagram

3.3 Recommendation Engine Architecture

A hybrid recommendation architecture ranks candidate videos by combining content affinity (vector similarity), collaborative filtering (social graph signals), and trending velocity (engagement decay). The signals flow as follows:

Input Signals:



Trending Score Calculation: To prevent viral outliers from overwhelming the feed, the engagement signal is log-compressed and exponentially decayed based on age:

```
trending_score = log(1 + likes + 0.5 * comments + 2 * shares) * e^(-0.05 * age_in_hours)
```

Key Components:

- `log(1 + engagement)`: Compresses viral outliers, maintaining feed diversity.
- `e^(-decay * age)`: Exponential decay ensures fresh content is promoted over old media.
- `shares * 2`: Weights shares heavily as they are strong indicators of user engagement.

3.4 Video Upload & Asynchronous Ingestion Flow

The sequence diagram below displays the lifecycle of a video upload. By using pre-signed URLs, the client uploads video bytes directly to S3. This keeps the API Gateway free from heavy file payloads and shifts processing workloads to an asynchronous Kafka/RabbitMQ queue consumed by FFmpeg workers.

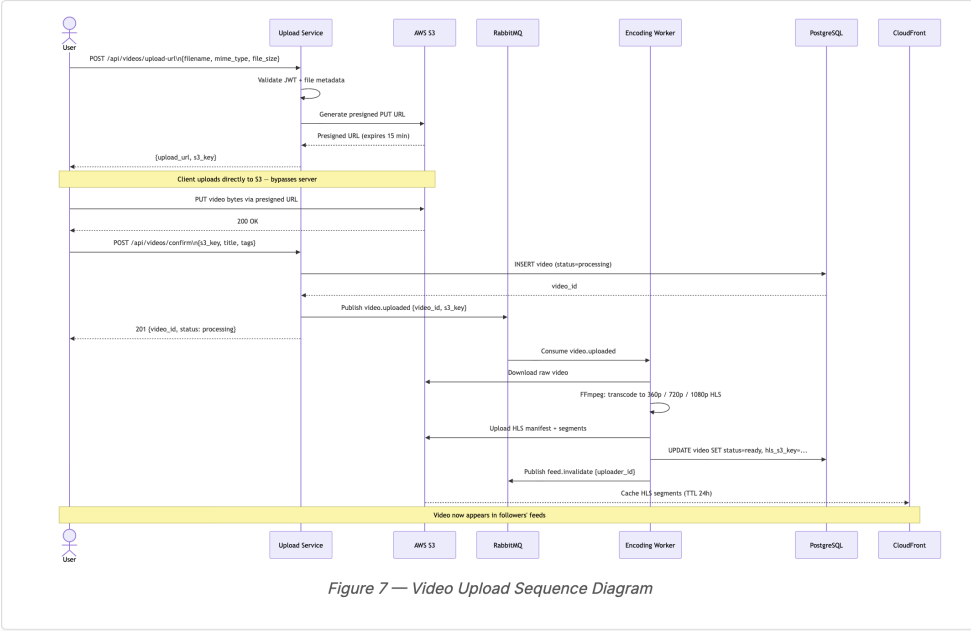


Figure 2 — Video Upload and Asynchronous Ingestion Sequence Diagram

3.5 Personalized Feed Retrieval Sequence

This flow highlights the real-time feed retrieval cycle. When a client requests a feed, the Video/Feed service queries the recommendation engine, fetches candidate IDs, formats the metadata via MySQL, and delivers CDN HLS links to the client.

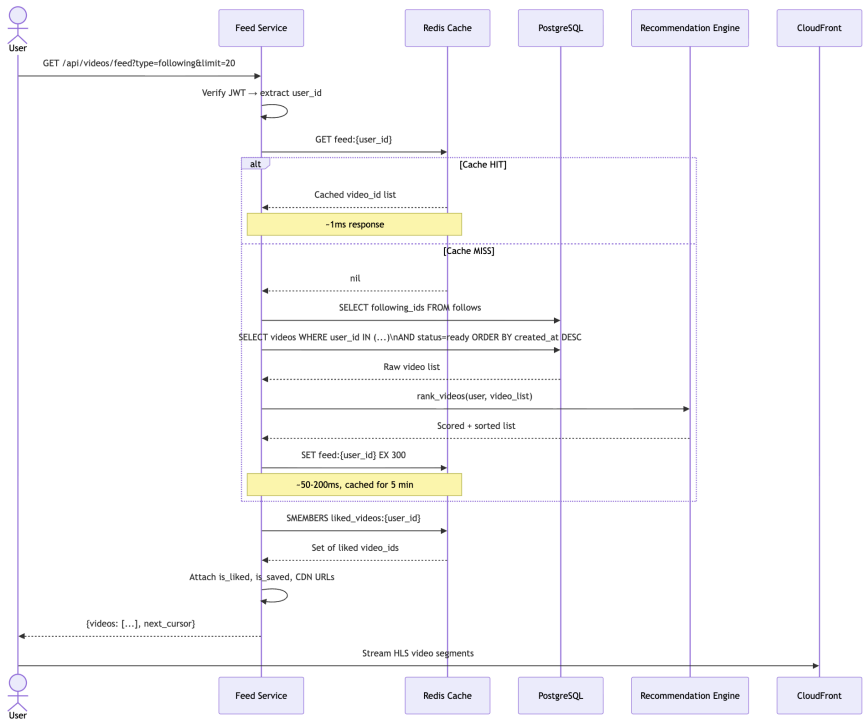


Figure 8 — Feed Request Sequence Diagram

Figure 3 — Personalized Feed Retrieval and Delivery Flow

4. Module Description

- **Auth Service:** Manages user accounts, handles credential hashing via bcrypt, and issues JSON Web Tokens (JWT) for stateless session verification.
- **Video Service:** Handles video uploads, manages metadata registration in MySQL, coordinates feed generation using cursor pagination, and returns CDN-cached URLs.
- **Interaction Service:** Manages likes, comments, and saves by incrementing atomic counters in Redis and dispatching event records.
- **Encoding Worker:** A background service that consumes transcoding events. It downloads raw video inputs from S3, splits them into HLS segments using FFmpeg, and uploads the manifest (.m3u8) back to S3.
- **Recommendation Service:** Orchestrates feed personalization. It runs candidate filtering and applies our hybrid scoring weights to recommend videos.

5. Database Design & Schema

A polyglot storage approach separates write workloads from read pipelines to optimize data delivery:

- **MySQL (Transactional / Sharded):** The source of truth for structured relational data. It stores normalized user records, profiles, following graphs, and video metadata. TikTok shards MySQL horizontally by hashing `user\_id` to scale writes across thousands of database instances.
- **Apache Cassandra (Interaction Log Storage):** An append-only store for interaction logs. It captures high-volume user activity streams (watch time logs, likes, shares, comments) without blocking relational reads.
- **Redis (Caching & Atomic Counter Storage):** Handles real-time feed cache lines (using key `feed:{user\_id}`) and manages view/like/share counts using atomic operations (INCR).

Database Type	Core Table / Entity	Schema Details & Attributes
MySQL	Users	user_id (PK/UUID), username, email, password_hash, created_at
MySQL	Videos	video_id (PK/UUID), user_id (FK), title, duration_sec, s3_url, created_at
Cassandra	Interaction_Logs	log_id (UUID), user_id (UUID), video_id (UUID), event_type, timestamp
Redis	Video_Counters	video:{id}:views, video:{id}:likes, video:{id}:shares (Atomic Hashes)

The database schema and relations are modeled below in the Entity Relationship Diagram:

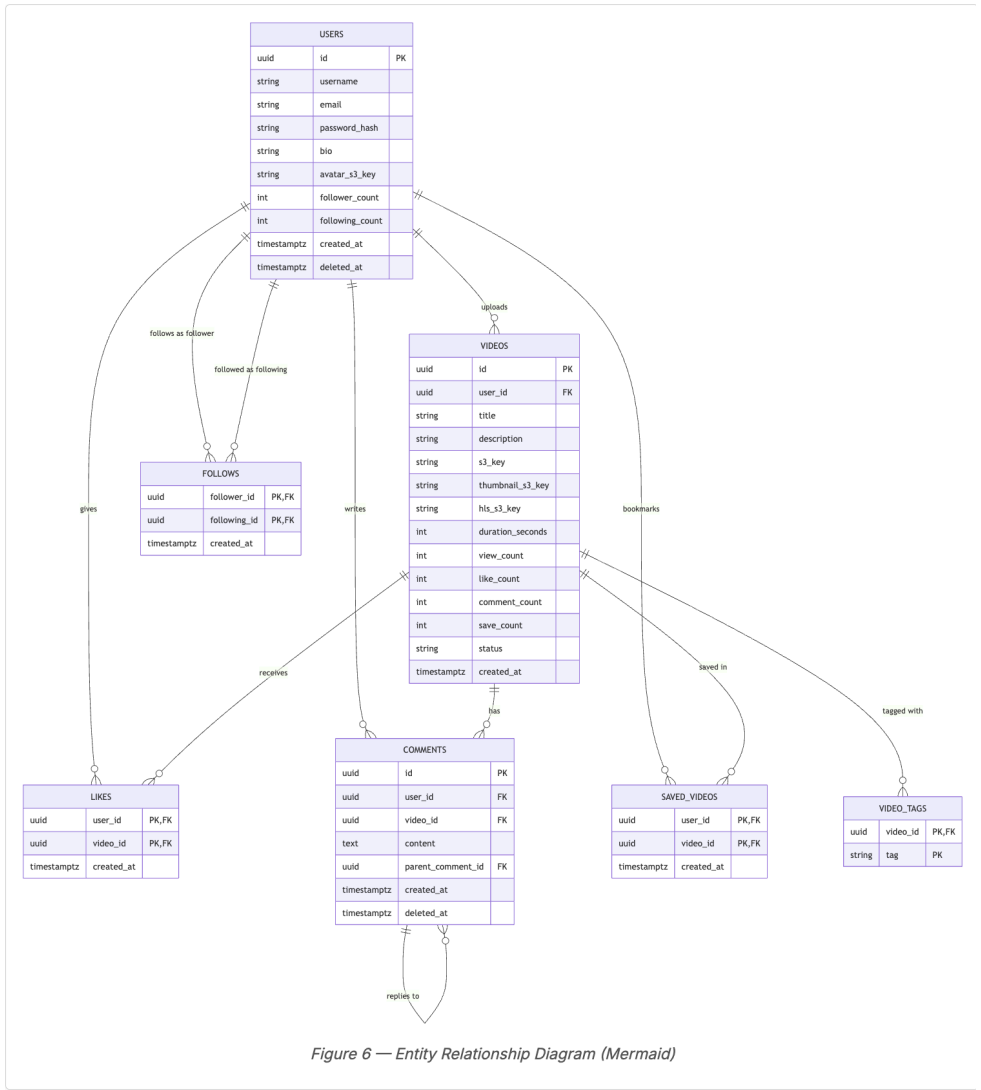


Figure 4 — MySQL Relational Schema Modeling User, Video and Follow entities

6. Technology Stack

- **API Gateway & Load Balancer:** NGINX
- **Application Framework:** Node.js / Express 4 (ES Modules)
- **Machine Learning & Simulation:** Python 3.10
- **Relational Storage:** MySQL (Sharded horizontally across instances)
- **NoSQL Ingestion Log Store:** Apache Cassandra 4
- **In-Memory Cache & Counters:** Redis 7
- **Message Broker:** RabbitMQ / Apache Kafka
- **Media Processing Engine:** FFmpeg segmenter
- **Storage Infrastructure:** AWS S3 & AWS CloudFront CDN

7. Core API Endpoints

The microservices expose RESTful interfaces to support client-side uploads and feed rendering:

Endpoint Target	HTTP Method	Endpoint Rationale & Operation
/api/video/upload	POST	Ingests filename and file size, returns an AWS S3 pre-signed URL, and registers a pending database record.
/api/feed/personalized	GET	Retrieves personalized feed listings using cursor-based pagination to prevent query offsets from skipping items.
/api/interaction/event	POST	Asynchronously logs a user interaction event to Cassandra and updates Redis counters.

8. Implementation Details: Recommendation Engine

The recommendation pipeline is simulated in a Python application that models the scoring algorithm. Rather than returning chronological updates, videos are ranked dynamically using three primary signals:

- Candidate Filtering:** A fast pass that narrows down the database to a subset of ~50 relevant videos based on hashtag matches, creator follows, and trending status.
- Scoring & Weighting:** Scores and ranks candidate videos using a custom weighted engagement model. The mathematical signals and their target weights are defined below:

Scoring Signal	Weight	Implementation Justification & Rationale
Watch Completion Rate	35%	The strongest signal of user interest. Prioritizes videos where users stay until the end, filtering out clickbait.
Like Velocity	20%	Calculates likes per hour. This highlights new trends and prevents old, viral videos from dominating the feed.
Share Rate	15%	Measures virality by tracking how often users share the video.
Content Affinity	15%	Calculates cosine similarity between a video's tags and the user's historical preference profile.
Social Boost	10%	Boosts scores for videos uploaded by creators whom the user follows.
Diversity Bonus	5%	Rewards videos with tags not recently seen in the user's feed, preventing filter bubbles.

9. Screenshots & Diagrams

This section consolidates the design diagrams and execution diagrams illustrating the platform's layout and transaction flow.

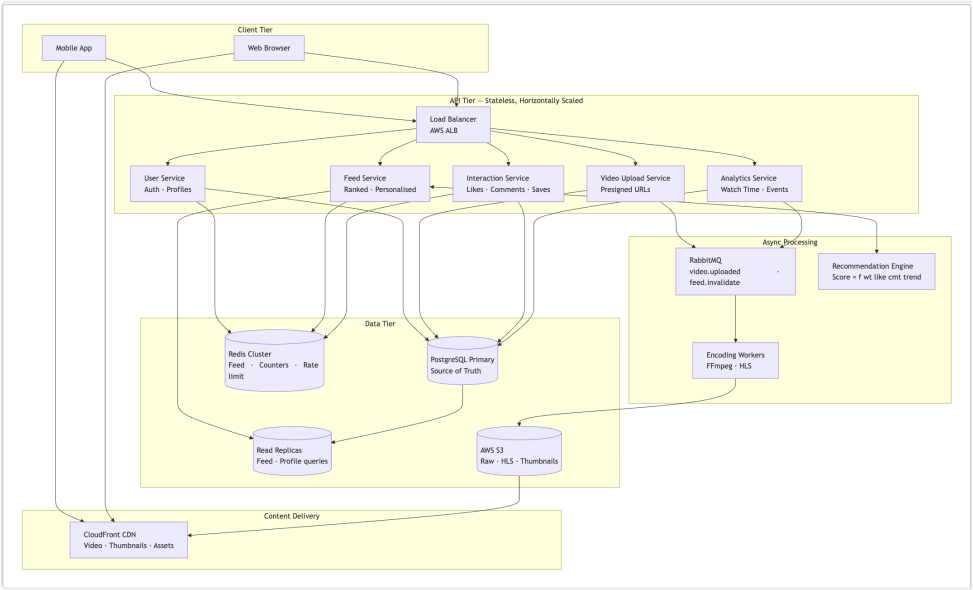


Figure 5 — Microservices System Architecture and CDN Edge Deployment

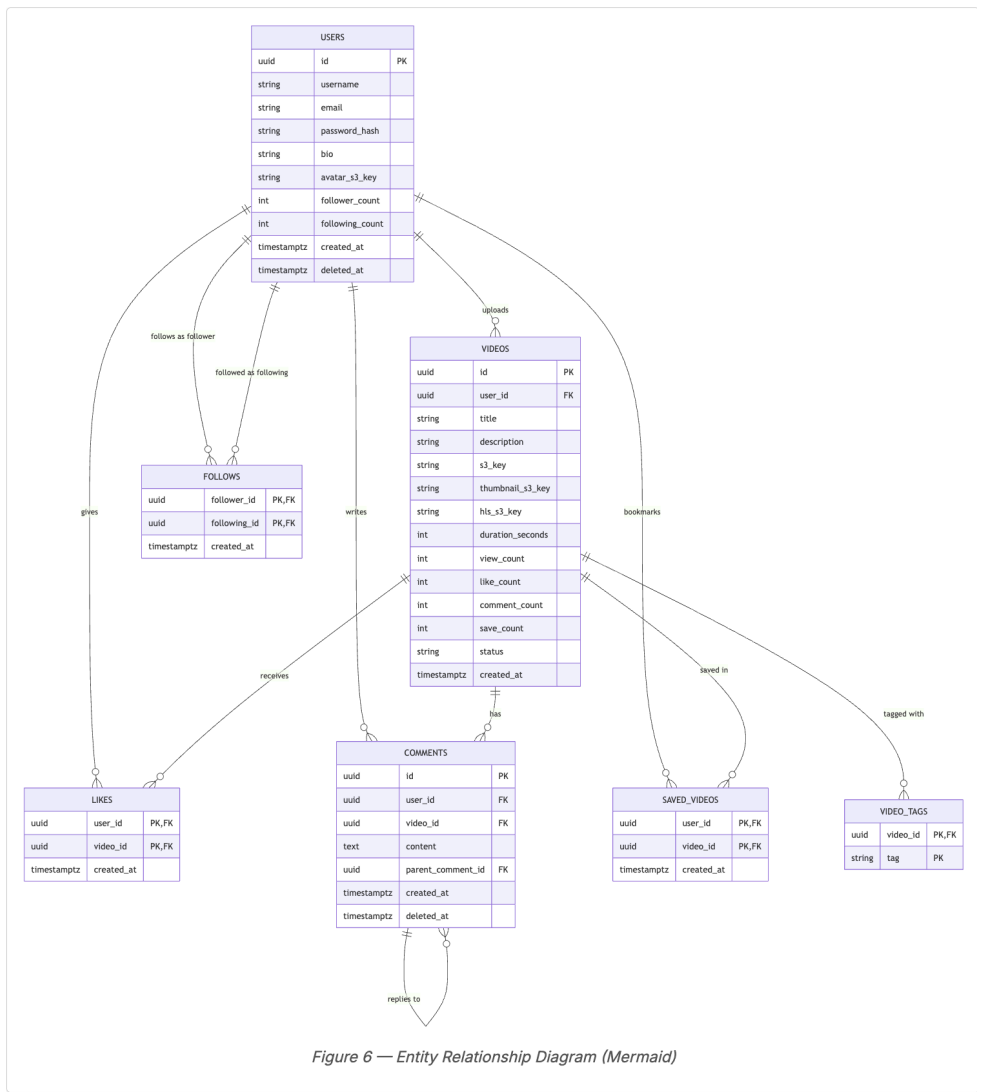


Figure 6 — Entity Relationship Diagram (Mermaid)

Figure 6 — MySQL Relational Schema Modeling User, Video and Follow entities

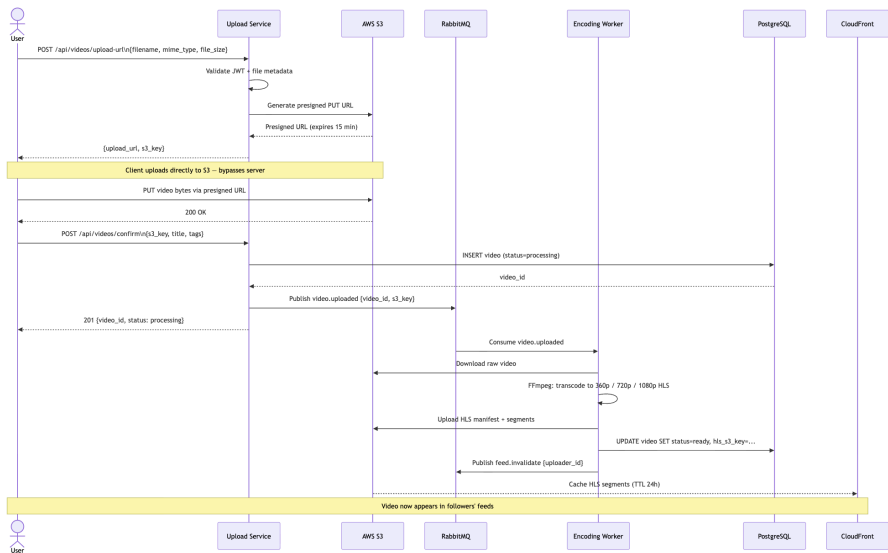


Figure 7 — Video Upload Sequence Diagram

Figure 7 — Asynchronous Video Ingestion and FFmpeg HLS Chunking Sequence

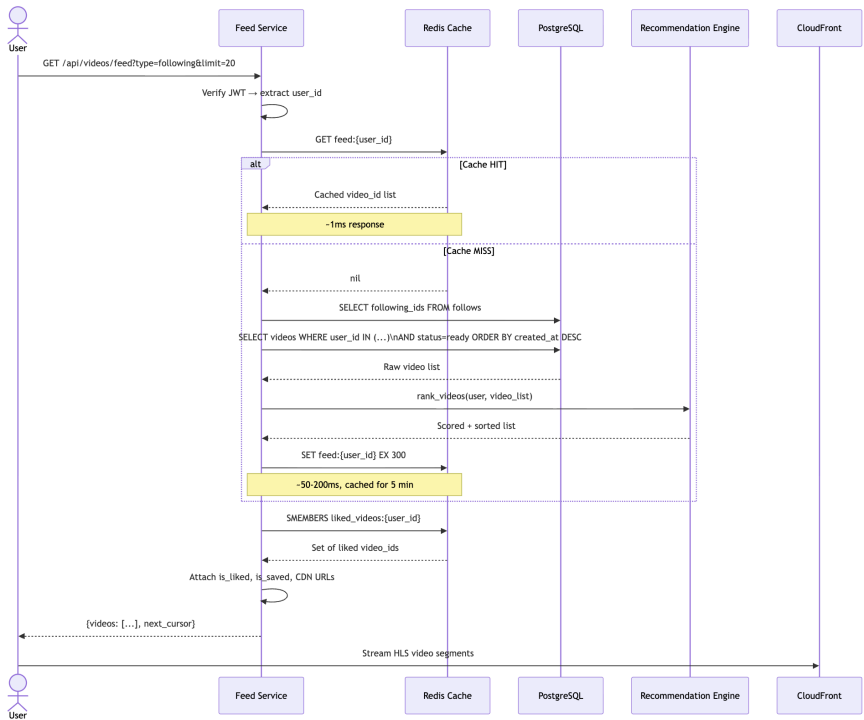


Figure 8 — Feed Request Sequence Diagram

Figure 8 — Dynamic Feed Generation and CDN Playback Delivery Flow

10. Future Scope

- **Low-Latency Live Streaming:** Implement WebRTC for sub-second live streaming and support RTMP ingestion alongside HLS pipelines.
- **AI-Powered Content Moderation:** Deploy computer vision models into the upload worker pipeline to automatically scan and flag violating content before it is published.
- **Deep Learning Personalization:** Replace the static weighted scoring algorithm with a Two-Tower Neural Network to capture complex user-video engagement relationships.
- **Creator Analytics Dashboard:** Integrate TimescaleDB for time-series logs, providing creators with real-time dashboards to analyze view velocity, retention rates, and follower demographics.
- **Interactive Duets & Stitching:** Develop server-side FFmpeg compositing pipelines to support side-by-side video merging and synchronized audio track mixing.
- **Semantic Video Search:** Deploy an Elasticsearch cluster using vector embeddings to enable semantic search across video titles, description metadata, and tags.

11. Conclusion

QuickClips demonstrates that the core architecture of a world-class short video platform — the same fundamental design used by TikTok, Instagram Reels, and YouTube Shorts — can be understood, designed, and partially implemented within a semester project timeframe.

Key architectural insights:

1. **Separate concerns at every layer** — storage, processing, delivery, and serving are distinct systems that scale independently.
2. **Cache aggressively** — Redis as the feed cache layer is what makes 200ms p99 latency achievable for millions of users.
3. **Write asynchronously** — decoupling video upload from encoding via message queues is what makes the system resilient to traffic spikes.
4. **Design for eventual consistency** — a `like_count` that's 30 seconds stale is completely acceptable; a broken auth system is not.
5. **The recommendation engine is the product** — the feed algorithm creates user retention, requiring careful signal design, not just a simple sort.

The Python simulation demonstrates that hybrid approaches — combining content-based filtering (user taste), collaborative filtering (social intelligence), and trending signals (platform-wide momentum) — produce more relevant feeds than any single strategy alone.